

Documentation

STEP 1

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 2

Markdown Content:

NumPy for numerical computations ,Pandas was used for data manipulation, Matplotlib for creating graphical representations, and Seaborn for generating advanced statistical visualizations. These libraries provide the foundation for preprocessing, analyzing, and interpreting the dataset effectively.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 3

Markdown Content:

SimpleImputer and KNNImputer were used to address missing data through statistical and machine learning-based techniques. The zscore function was utilized to detect outliers, while winsorize was applied to limit the effect of extreme values. These preprocessing steps improve data quality, enhance model performance, and ensure reliable analytical results.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 4

Code:

```
df = pd.read_csv("/content/RealEstate_HousePrice_Dataset_4200 - 
RealEstate_HousePrice_Dataset_4200.csv.csv")
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 5

Markdown Content:

The dataset was loaded into a Pandas DataFrame to enable data analysis, preprocessing, and visualization.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 6

Code:

```
df.head()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 7

Markdown Content:

The first few records were examined to understand the structure of the dataset and identify the available features.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 8

Code:

```
df.tail()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 9

Code:

```
df.shape
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 10

Markdown Content:

The total number of rows and columns was identified to understand the size and dimensionality of the dataset.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 11

Code:

```
df.info()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 12

Markdown Content:

The dataset information was examined to understand the data types, identify missing values, and estimate memory usage.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 13

Code:

```
df.columns
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 14

Code:

```
df.isnull().sum()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 15

Markdown Content:

lays the number of missing (null) values present in each column of the dataset.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 16

Code:

```
df.duplicated().sum()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 17

Markdown Content:

This code is used to check sum of duplicate .

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 18

Markdown Content:

Duplicate records were checked to ensure data quality. Removing duplicate entries helps prevent biased model training.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 19

Code:

```
X = df.drop("house_price_inr", axis=1)
```

```
y = df["house_price_inr"]
```

```
print("Feature Shape :", X.shape)
```

```
print("Target Shape :", y.shape)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 20

Markdown Content:

This code used to check seprate the Feature shape and target shape.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 21

Code:

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(12,6))
```

```
sns.heatmap(
    df.isnull(),
    cbar=False,
    cmap='viridis'
)

plt.title("Missing Values Heatmap")

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 22

Markdown Content:

The heatmap visually highlights missing values in the dataset. Yellow/light areas indicate missing values, while dark areas indicate available data.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 23

Code:

```
import matplotlib.pyplot as plt

df.hist(
    figsize=(20, 15),
    bins=30,
    color='mediumpurple',
    edgecolor='black'
)

plt.tight_layout()
plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 24

Markdown Content:

This code generates histograms for all numerical columns in the dataset. The histograms help understand the distribution, frequency, spread, and possible outliers of each feature before building the machine learning model.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 25

Code:

```
plt.figure(figsize=(10,6))

sns.histplot(
    df["house_price_inr"],
    bins=40,
    kde=True,
    color="coral"
)

plt.title("Distribution of House Prices")
plt.xlabel("House Price (INR)")
plt.ylabel("Count")

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 26

Markdown Content:

This visualization provides valuable insights into the distribution of house prices in the dataset. It helps identify common price ranges, detect skewness or outliers, and assess whether the target variable follows a normal distribution. These insights are useful for selecting appropriate preprocessing techniques and improving the performance of the house price prediction model.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 27

Code:

```

import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

numerical_columns = df.select_dtypes(include=np.number).columns

plt.figure(figsize=(20,20))

for i, column in enumerate(numerical_columns):

    plt.subplot(4,4,i+1)

    sns.boxplot(
        y=df[column],
        color="royalblue"
    )

    plt.title(column, fontsize=12, fontweight="bold")

plt.tight_layout()

plt.show()

```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 28

Markdown Content:

This code creates boxplots for all numerical columns in the dataset. The boxplots help visualize the distribution, median, spread, and potential outliers of each feature, allowing better understanding of the data before model development.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 29

Code:

```

corr = df.corr(numeric_only=True)

plt.figure(figsize=(15,10))

sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",

```



```
        fmt=".2f"
    )

plt.title("Correlation Heatmap")

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 30

Markdown Content:

This code calculates the correlation between all numerical features in the dataset and displays the results as a heatmap. The correlation matrix measures the strength and direction of the relationship between pairs of variables, with values ranging from -1 to +1. Positive values indicate that two variables increase together, while negative values indicate an inverse relationship. The heatmap uses different colors to represent the correlation strength, making it easier to identify highly related features, potential multicollinearity, and the variables that have the strongest relationship with the target variable (house_price_inr). This analysis helps in feature selection and improves the understanding of the dataset before building machine learning models.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 31

Code:

```
correlation = corr["house_price_inr"].sort_values(ascending=False)

print(correlation)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 32

Markdown Content:

This code extracts the correlation values between the target variable (`house_price_inr`) and all other numerical features in the dataset. The correlation values are then sorted in descending order, allowing the features with the strongest positive relationship to appear first. Printing this sorted list makes it easy to identify which variables have the greatest influence on house prices and which have little or negative impact. This analysis is valuable for feature selection, as highly correlated features are generally more useful for building accurate machine learning models.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 33

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))

sns.countplot(
    x="has_garage",
    data=df,
    hue="has_garage",
    palette=["royalblue", "coral"],
    legend=False
)

plt.title("Properties with Garage", fontsize=14, fontweight="bold")
plt.xlabel("Has Garage")
plt.ylabel("Count")

plt.grid(axis="y", alpha=0.3)

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 34

Markdown Content:

This code creates a count plot for the `has_garage` feature to show the number of properties with and without a garage. It helps analyze the distribution of this categorical variable and understand its potential impact on house prices.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 35

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))

sns.countplot(
    x="has_pool",
    data=df,
    hue="has_pool",
    palette=["royalblue", "coral"],
    legend=False
)

plt.title("Properties with Pool", fontsize=14, fontweight="bold")
plt.xlabel("Has Pool")
plt.ylabel("Count")

plt.grid(axis="y", alpha=0.3)

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 36

Markdown Content:

This code generates a count plot for the `has_pool` feature to compare the number of properties with and without a swimming pool. It helps understand the distribution of this categorical feature during exploratory data analysis (EDA).

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 37

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
plt.figure(figsize=(8,5))

sns.barplot(
    data=df,
    x="bedrooms",
    y="house_price_inr",
    hue="bedrooms",
    palette="Set2",
    legend=False
)

plt.title("Average House Price by Bedrooms", fontsize=14, fontweight="bold")
plt.xlabel("Bedrooms", fontsize=12)
plt.ylabel("Average House Price (INR)", fontsize=12)

plt.grid(axis="y", alpha=0.3)

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 38

Markdown Content:

This code creates a bar plot to compare the average house price for properties with different numbers of bedrooms. Each bar represents the mean house price for a specific bedroom category, allowing an easy comparison of how property prices vary as the number of bedrooms increases. Different colors are used to distinguish each bedroom category, making the chart more visually appealing. This analysis helps determine whether the number of bedrooms has a significant impact on house prices and provides valuable insights for feature selection in the predictive model.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 39

Code:

```
print("Missing Values in Each Column:\n")
print(df.isnull().sum())
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 40

Markdown Content:

This code checks for missing values in every column of the dataset and displays the total count of null values. It helps identify incomplete data that may require preprocessing before model development.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 41

Code:

```
binary_columns = []  
  
df.head()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 42

Markdown Content:

This code creates an empty list to store binary feature names and displays the first five rows of the dataset. It helps inspect the data and identify columns that contain binary values (0 and 1).

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 43

Code:

```
X = df.drop("house_price_inr", axis=1)  
  
y = df["house_price_inr"]  
  
print("Feature Shape :", X.shape)  
print("Target Shape :", y.shape)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 44

Markdown Content:

This code separates the dataset into input features (X) and the target variable (y). The feature matrix contains all independent variables, while the target vector contains the house prices to be predicted. The shapes of both are printed to verify the data preparation.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 45

Code:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training Features :", X_train.shape)
print("Testing Features  :", X_test.shape)
print("Training Target   :", y_train.shape)
print("Testing Target    :", y_test.shape)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 46

Markdown Content:

This code divides the dataset into training and testing sets using an 80:20 ratio. The training data is used to build the machine learning model, while the testing data is used to evaluate its prediction performance. The shapes of the resulting datasets are displayed for verification.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 47

Code:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

print("Feature Scaling Completed Successfully!")
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 48

Markdown Content:

This code standardizes the input features using StandardScaler. The scaler is fitted on the training data and then applied to both the training and testing datasets. Feature scaling improves model performance by ensuring all variables are measured on the same scale.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 49

Code:

```
scaled_df = pd.DataFrame(
    X_train_scaled,
    columns=X_train.columns
)

scaled_df.head()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 50

Markdown Content:

This step converts the standardized training data into a well-structured DataFrame with the original feature names, making the scaled data easier to inspect and interpret. Displaying the first five rows confirms that feature scaling has been successfully applied and that the dataset is properly formatted for machine learning model training

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 51

Code:

```
from sklearn.linear_model import Ridge
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 52

Code:

```
ridge_model = Ridge(alpha=1.0)

ridge_model.fit(X_train_scaled, y_train)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 53

Markdown Content:

This code initializes a Ridge Regression model with a regularization parameter of 1.0 and trains it using the standardized training dataset. Ridge Regression helps reduce overfitting and improves the model's ability to generalize to unseen data.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 54

Code:

```
ridge_train_pred = ridge_model.predict(X_train_scaled)

ridge_test_pred = ridge_model.predict(X_test_scaled)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 55

Markdown Content:

This code generates predicted house prices for both the training and testing datasets using the trained Ridge Regression model. These predictions are used to evaluate the model's accuracy and overall performance.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 56

Code:

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

ridge_mae = mean_absolute_error(y_test, ridge_test_pred)

ridge_mse = mean_squared_error(y_test, ridge_test_pred)

ridge_rmse = np.sqrt(ridge_mse)

ridge_r2 = r2_score(y_test, ridge_test_pred)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 57

Markdown Content:

This code evaluates the performance of the Ridge Regression model by calculating four widely used regression evaluation metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 Score. These metrics compare the actual house prices with the predicted values to measure the model's prediction accuracy. Lower values of MAE, MSE, and RMSE indicate fewer prediction errors, while an R^2 Score closer to 1 indicates that the model explains a larger proportion of the variation in house prices. Together, these metrics provide a comprehensive assessment of the model's performance.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 58

Code:

```
train_score = ridge_model.score(X_train_scaled, y_train)
```

```
test_score = ridge_model.score(X_test_scaled, y_test)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 59

Markdown Content:

This code calculates the R^2 score (coefficient of determination) of the trained Ridge Regression model on both the training and testing datasets. The training score measures how well the model has learned the patterns from the training data, while the testing score evaluates how effectively the model predicts house prices for new, unseen data. Comparing these two scores helps determine whether the model is well-generalized, overfitting, or underfitting. Ideally, both scores should be high and close to each other, indicating that the model performs consistently on both known and unseen data.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 60

Code:

```

ridge_coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": ridge_model.coef_
})

ridge_coefficients.sort_values(
    by="Coefficient",
    ascending=False
)

```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 61

Markdown Content:

This code extracts the feature coefficients learned by the trained Ridge Regression model and organizes them into a structured table. Each coefficient represents the influence of a feature on the predicted house price. A positive coefficient indicates that an increase in the feature value tends to increase the predicted house price, while a negative coefficient suggests that an increase in the feature value tends to decrease the predicted house price. Finally, the coefficients are sorted in descending order, making it easy to identify the most influential features in the model.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 62

Code:

```

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,6))

sns.barplot(
    data=ridge_coefficients.sort_values("Coefficient"),
    x="Coefficient",
    y="Feature",
    hue="Feature",
    palette="Spectral",
    legend=False
)

```

```
plt.title("Ridge Regression Feature Coefficients", fontsize=16, fontweight="bold")
plt.xlabel("Coefficient", fontsize=13)
plt.ylabel("Feature", fontsize=13)

plt.grid(axis="x", linestyle="--", alpha=0.3)

plt.tight_layout()
plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 63

Markdown Content:

This code creates a horizontal bar chart to visualize the coefficients of the trained Ridge Regression model. Each bar represents the importance and direction of a feature's contribution to house price prediction. Features with positive coefficients increase the predicted house price, while features with negative coefficients decrease it. The coefficients are sorted in ascending order so that the least influential negative features appear at the bottom and the strongest positive features appear at the top. This visualization makes it easier to identify the most significant factors affecting house prices and improves the interpretability of the machine learning model.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 64

Code:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

plt.scatter(
    y_test,
    ridge_test_pred,
    color="dodgerblue",
    edgecolors="black",
    alpha=0.7,
    s=60
)

plt.xlabel("Actual Price", fontsize=12)
plt.ylabel("Predicted Price", fontsize=12)
plt.title("Actual vs Predicted House Prices (Ridge Regression)", fontsize=14,
fontweight="bold")
```

```
plt.grid(True, linestyle="--", alpha=0.4)
```

```
plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 65

Markdown Content:

This scatter plot visually evaluates the performance of the Ridge Regression model by comparing actual and predicted house prices. A strong concentration of points near the imaginary diagonal line indicates high prediction accuracy and good model generalization. Any points that lie far from this line highlight prediction errors or outliers, making this visualization an effective tool for assessing the reliability and overall performance of the house price prediction model.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 66

Code:

```
from sklearn.linear_model import Lasso
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 67

Code:

```
lasso_model = Lasso(alpha=1.0) # Initialize the Lasso model  
  
lasso_model.fit(X_train_scaled, y_train) # Fit the Lasso model  
  
lasso_train_pred = lasso_model.predict(X_train_scaled)  
  
lasso_test_pred = lasso_model.predict(X_test_scaled)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 68

Markdown Content:

This step initializes and trains the Lasso Regression model using the standardized training data and generates predictions for both the training and testing datasets. By applying L1 regularization, Lasso Regression reduces overfitting while automatically identifying the most important features for house price prediction. The generated predictions will be used in the next step to evaluate the model's performance using metrics such as MAE, MSE, RMSE, and R² Score.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 69

Code:

```
lasso_mae = mean_absolute_error(y_test, lasso_test_pred)

lasso_mse = mean_squared_error(y_test, lasso_test_pred)

lasso_rmse = np.sqrt(lasso_mse)

lasso_r2 = r2_score(y_test, lasso_test_pred)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 70

Markdown Content:

This code evaluates the performance of the Lasso Regression model by calculating four important regression evaluation metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R² Score. These metrics compare the actual house prices with the values predicted by the Lasso Regression model to determine its prediction accuracy. Lower values of MAE, MSE, and RMSE indicate smaller prediction errors, while an R² Score closer to 1 indicates that the model explains a larger proportion of the variation in house prices. These evaluation metrics are essential for assessing the effectiveness of the Lasso Regression model and comparing its performance with other models such as Ridge Regression.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 71

Code:

```
lasso_train_score = lasso_model.score(X_train_scaled, y_train)

lasso_test_score = lasso_model.score(X_test_scaled, y_test)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 72

Markdown Content:

This step evaluates the Lasso Regression model by calculating the R^2 scores for both the training and testing datasets. Comparing these scores helps assess whether the model has learned meaningful patterns without overfitting. A high and similar R^2 score on both datasets indicates that the model is reliable, generalizes well to unseen data, and is suitable for accurate house price prediction.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 73

Code:

```
lasso_coefficients = pd.DataFrame({

    "Feature": X.columns,

    "Coefficient": lasso_model.coef_

})

lasso_coefficients.sort_values(
    by="Coefficient",
    ascending=False
)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 74

Markdown Content:

This code extracts the feature coefficients learned by the trained Lasso Regression model and stores them in a Pandas DataFrame. Each coefficient indicates the importance and direction of a feature's influence on house price prediction. Features with positive coefficients contribute to increasing the predicted house price, while negative coefficients contribute to decreasing it. One of the key characteristics of Lasso Regression is that it can assign a coefficient of zero to less important features, effectively removing them from the model. Finally, the coefficients are sorted in descending order to identify the most influential features.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 75

Code:

```
plt.figure(figsize=(12,6))

sns.barplot(
    data=lasso_coefficients.sort_values("Coefficient"),
    x="Coefficient",
    y="Feature",
    palette="viridis"    # try: "magma", "plasma", "coolwarm"
)

plt.title("Lasso Regression Feature Coefficients")

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 76

Markdown Content:

This visualization provides a clear representation of the feature coefficients learned by the Lasso Regression model. By displaying the magnitude and direction of each coefficient, it helps identify the most influential variables affecting house prices. Additionally, it highlights Lasso Regression's ability to perform automatic feature selection by assigning zero coefficients to less important features, resulting in a simpler, more interpretable, and efficient predictive model.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 77

Code:

```
plt.figure(figsize=(8,6))

plt.scatter(
    y_test,
    lasso_test_pred,
    alpha=0.6,
    color="royalblue",
    edgecolor="black",
    s=60
)

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color="red",
    linestyle="--",
    linewidth=2,
    label="Perfect Prediction"
)

plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title("Actual vs Predicted (Lasso Regression)")

plt.legend()
plt.grid(True, linestyle="--", alpha=0.4)

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 78

Markdown Content:

This code creates a scatter plot to compare the actual house prices with the house prices predicted by the Lasso Regression model. Each point on the graph represents one property from the testing dataset, where the x-axis shows the actual house price and the y-axis shows the predicted house price. If the model predicts accurately, most points will lie close to an imaginary 45-degree diagonal line, indicating that the

predicted values closely match the actual values. This visualization helps assess the prediction accuracy of the Lasso Regression model and identify any outliers or large prediction errors.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 79

Code:

```
lasso_residuals = y_test - lasso_test_pred

plt.figure(figsize=(8,6))

sns.scatterplot(
    x=lasso_test_pred,
    y=lasso_residuals,
    color="teal",
    alpha=0.6,
    edgecolor="black",
    s=70
)

plt.axhline(0, color='red', linestyle='--', linewidth=2)

plt.xlabel("Predicted Price")
plt.ylabel("Residual (Actual - Predicted)")
plt.title("Residual Plot (Lasso Regression)")

plt.grid(True, linestyle="--", alpha=0.4)

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 80

Markdown Content:

This code calculates the residuals (prediction errors) of the Lasso Regression model and visualizes them using a residual scatter plot. A residual is the difference between the actual house price and the predicted house price. The plot displays the predicted house prices on the x-axis and the corresponding residuals on the y-axis. A horizontal reference line is drawn at zero residual, representing perfect predictions. If the residuals are randomly scattered around this line without any clear pattern, it indicates that the Lasso Regression model fits the data well. However, visible patterns or systematic trends may suggest that the model is missing important relationships or assumptions.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 81

Code:

```
comparison = pd.DataFrame({  
    "Model": ["Ridge Regression", "Lasso Regression"],  
    "MAE": [ridge_mae, lasso_mae],  
    "MSE": [ridge_mse, lasso_mse],  
    "RMSE": [ridge_rmse, lasso_rmse],  
    "R2 Score": [ridge_r2, lasso_r2]  
})  
  
comparison
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 82

Markdown Content:

This code creates a comparison table that summarizes the performance of the Ridge Regression and Lasso Regression models using four important evaluation metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 Score. By placing the results of both models side by side, the table makes it easy to compare their prediction accuracy and overall performance. This comparison helps identify which regression model is more suitable for the house price prediction task based on lower error values and a higher R^2 Score.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 83

Code:

```
comparison.set_index("Model")[["MAE", "RMSE"]].plot(  
    kind="bar",  
    figsize=(8,5)
```

```
)  
  
plt.title("Ridge vs Lasso Performance")  
  
plt.ylabel("Error")  
  
plt.xticks(rotation=0)  
  
plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 84

Markdown Content:

This code creates a bar chart to compare the Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) of the Ridge Regression and Lasso Regression models. These two metrics measure the prediction error of each model, making it easy to visually compare their performance. Lower bars indicate smaller prediction errors and therefore better model accuracy. This visualization helps quickly identify which regression model performs better in predicting house prices.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 85

Code:

```
ridge_zero = np.sum(ridge_model.coef_ == 0)  
  
lasso_zero = np.sum(lasso_model.coef_ == 0)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 86

Markdown Content:

This code calculates the number of features whose coefficients are exactly zero in both the Ridge Regression and Lasso Regression models. A coefficient of zero means that the corresponding feature does not contribute to the model's predictions. Ridge Regression typically shrinks coefficients toward zero but rarely makes them exactly zero, whereas Lasso Regression can set some coefficients exactly to zero through L1 regularization, effectively performing automatic feature selection. Comparing the number of zero coefficients helps understand how each model handles feature importance and model complexity.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 87

Code:

```
from sklearn.model_selection import GridSearchCV
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 88

Code:

```
alpha_values = {  
    "alpha": [0.001, 0.01, 0.1, 1, 10, 25, 50, 75, 100]  
}  
  
print(alpha_values)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 89

Markdown Content:

This code creates a dictionary containing a list of different alpha (α) values that will be used to tune the regularization strength of regression models such as Ridge Regression and Lasso Regression. The alpha parameter controls the amount of regularization applied during model training. Smaller alpha values apply weaker regularization, allowing the model to fit the training data more closely, while larger alpha values apply stronger regularization, reducing model complexity and helping prevent overfitting. Finally, the dictionary is printed to display all the selected alpha values that can be used for hyperparameter tuning.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 90

Code:

```
ridge_grid = GridSearchCV(
    estimator=Ridge(),
    param_grid=alpha_values,
    scoring="r2",
    cv=5,
    n_jobs=-1
)
ridge_grid.fit(X_train_scaled, y_train)
print("Best Alpha :", ridge_grid.best_params_)
print("Best CV Score :", ridge_grid.best_score_)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 91

Markdown Content:

This code uses GridSearchCV to find the optimal alpha value for the Ridge Regression model. It evaluates multiple alpha values using 5-fold cross-validation and selects the one with the highest R² Score, improving the model's prediction performance.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 92

Code:

```
best_ridge = ridge_grid.best_estimator_
ridge_prediction = best_ridge.predict(X_test_scaled)
```

```
ridge_mae_best = mean_absolute_error(y_test,ridge_prediction)

ridge_mse_best = mean_squared_error(y_test,ridge_prediction)

ridge_rmse_best = np.sqrt(ridge_mse_best)

ridge_r2_best = r2_score(y_test,ridge_prediction)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 93

Markdown Content:

This code takes the best Ridge Regression model (selected using GridSearchCV) and evaluates its performance on the test dataset. After finding the optimal alpha value earlier, the code retrieves the best-trained model (best_estimator_) and uses it to predict house prices for unseen data. Then, it calculates important evaluation metrics—MAE, MSE, RMSE, and R² Score—to measure how accurately the optimized model performs. These metrics help confirm whether hyperparameter tuning improved the model compared to the default Ridge Regression model.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 94

Code:

```
lasso_grid = GridSearchCV(

    estimator=Lasso(max_iter=10000),

    param_grid=alpha_values,

    scoring="r2",

    cv=5,

    n_jobs=-1

)

lasso_grid.fit(X_train_scaled,y_train)

print("Best Alpha :",lasso_grid.best_params_)
```

```
print("Best CV Score :",lasso_grid.best_score_)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 95

Markdown Content:

This code performs hyperparameter tuning for the Lasso Regression model using GridSearchCV. It automatically tests multiple values of the alpha parameter to find the best regularization strength that gives the highest prediction accuracy. The evaluation is done using 5-fold cross-validation with R² Score as the performance metric. The goal is to identify the optimal alpha value that improves model performance while also controlling feature selection (since Lasso can eliminate less important features). Finally, the code prints the best alpha value and the corresponding cross-validation score.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 96

Code:

```
best_lasso = lasso_grid.best_estimator_  
  
lasso_prediction = best_lasso.predict(X_test_scaled)  
  
lasso_mae_best = mean_absolute_error(y_test,lasso_prediction)  
  
lasso_mse_best = mean_squared_error(y_test,lasso_prediction)  
  
lasso_rmse_best = np.sqrt(lasso_mse_best)  
  
lasso_r2_best = r2_score(y_test,lasso_prediction)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 97

Markdown Content:

This step evaluates the optimized Lasso Regression model obtained from GridSearchCV. By using the best alpha value, the model generates more accurate predictions on unseen test data. The evaluation metrics (MAE, MSE, RMSE, and R^2 Score) confirm the effectiveness of hyperparameter tuning and help compare Lasso with Ridge Regression to identify the best-performing model for house price prediction.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 98

Code:

```
tuned_models = pd.DataFrame({

    "Model":[
        "Best Ridge",
        "Best Lasso"
    ],

    "Best Alpha":[
        ridge_grid.best_params_["alpha"],
        lasso_grid.best_params_["alpha"]
    ],

    "MAE":[
        ridge_mae_best,
        lasso_mae_best
    ],

    "MSE":[
        ridge_mse_best,
        lasso_mse_best
    ],

    "RMSE":[
        ridge_rmse_best,
        lasso_rmse_best
    ],

    "R2 Score":[
        ridge_r2_best,
```

```

        lasso_r2_best
    ]
})

```

tuned_models

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 99

Markdown Content:

This step creates a final evaluation summary table comparing the best Ridge and Lasso Regression models after tuning. It includes optimal alpha values and all key performance metrics, allowing a clear and structured comparison. This helps determine which model performs better for house price prediction and provides a strong basis for selecting the final model for real-world use or deployment.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 100

Code:

```

plt.figure(figsize=(6,4))

plt.bar(
    ["Ridge", "Lasso"],
    [
        ridge_grid.best_params_["alpha"],
        lasso_grid.best_params_["alpha"]
    ],
    color=["royalblue", "orange"], # added colors
    edgecolor="black",
    width=0.5
)

plt.title("Best Alpha Values")
plt.ylabel("Alpha Value")

plt.grid(axis='y', linestyle='--', alpha=0.4)

plt.show()

```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 101

Markdown Content:

This code visualizes the best alpha values selected for Ridge and Lasso Regression models after tuning. It helps compare the optimal regularization strength for both models.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 102

Code:

```
if ridge_r2_best > lasso_r2_best:

    print("Best Regularized Model : Ridge Regression")

else:

    print("Best Regularized Model : Lasso Regression")
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 103

Markdown Content:

Ridge and Lasso Regression were compared using R^2 score to find the best model. The model with the higher R^2 is selected as the best performer. Ridge helps reduce overfitting while keeping all features, whereas Lasso also performs feature selection by removing less important features. Overall, the best model is the one that gives higher accuracy and better generalization on unseen data.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 104

Code:

```
from sklearn.model_selection import (
    KFold,
    StratifiedKFold,
    LeaveOneOut,
    TimeSeriesSplit,
    cross_val_score
)

print("Cross Validation Libraries Imported Successfully!")
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 105

Markdown Content:

This code imports different cross-validation techniques such as KFold, StratifiedKFold, LeaveOneOut, TimeSeriesSplit, and cross_val_score from scikit-learn. These methods are used to evaluate machine learning models more reliably by testing them on different splits of the dataset. The successful import confirms that all tools required for model validation are ready to use.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 106

Code:

```
kfold = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

ridge_kfold = cross_val_score(
    best_ridge,
    X_train_scaled,
    y_train,
    cv=kfold,
    scoring="r2"
)

print("K-Fold Scores:\n", ridge_kfold)

print("\nAverage R2 Score:", ridge_kfold.mean())
```

```
print("Standard Deviation:", ridge_kfold.std())
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 107

Markdown Content:

This code applies 5-Fold Cross Validation (KFold) on the Ridge Regression model to evaluate its performance more reliably. The dataset is split into 5 parts, and the model is trained and tested multiple times to get different R^2 scores. The final output includes individual fold scores, along with the average R^2 score and standard deviation, which helps measure the model's accuracy and stability.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 108

Code:

```
y_binned = pd.qcut(
    y_train,
    q=5,
    labels=False
)

print(y_binned.value_counts())
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 109

Markdown Content:

This code converts the continuous target variable (`y_train`) into 5 equal-sized bins using `qcut`, creating categorical groups. It is mainly used for better distribution analysis or for techniques like stratified sampling. The output shows how many data points fall into each bin, helping to understand the distribution of the target variable.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 110

Code:

```
skf = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

stratified_scores = []

for train_index, valid_index in skf.split(X_train_scaled, y_binned):

    X_tr = X_train_scaled[train_index]
    X_val = X_train_scaled[valid_index]

    y_tr = y_train.iloc[train_index]
    y_val = y_train.iloc[valid_index]

    best_ridge.fit(X_tr, y_tr)

    score = best_ridge.score(X_val, y_val)

    stratified_scores.append(score)

print("Stratified K-Fold Scores:\n")

print(stratified_scores)

print("\nAverage R2 Score:", np.mean(stratified_scores))
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 111

Markdown Content:

This code applies Stratified K-Fold Cross Validation on the Ridge Regression model by splitting the dataset into 5 folds while maintaining a balanced distribution of the target variable (using `y_binned`). The model is trained and validated on each fold, and the R^2 scores are collected to evaluate performance. Finally, the average R^2 score is calculated to measure the model's overall accuracy and stability across different data splits.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 112

Code:

```
tscv = TimeSeriesSplit(n_splits=5)

time_scores = cross_val_score(
    best_ridge,
    X_train_scaled,
    y_train,
    cv=tscv,
    scoring="r2"
)

print("Time Series Scores:\n")

print(time_scores)

print("\nAverage Score:", time_scores.mean())
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 113

Markdown Content:

This code evaluates a Ridge Regression model using Time Series Cross Validation (TimeSeriesSplit). The dataset is split in a sequential manner to respect time order, making it suitable for time-dependent data. The model is trained and tested across 5 splits, and the R^2 scores are calculated for each split. Finally, the average score is used to measure the overall predictive performance of the model on time-based data.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 114

Code:

```
best_cv = cv_results.loc[
    cv_results["Average R2"].idxmax()
]

print(best_cv)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 115

Markdown Content:

This code selects the best-performing model from cross-validation results by finding the row with the highest average R^2 score. It identifies the model that performs best on unseen data and prints its complete details. This helps in final model selection based on overall validation performance.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 116

Code:

```
from sklearn.tree import DecisionTreeRegressor

print("Decision Tree Imported Successfully!")
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 117

Markdown Content:

Import decision tree regressor completely.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 118

Code:

```
decision_tree = DecisionTreeRegressor(
    random_state=42
)
```



```
decision_tree.fit(X_train, y_train)

print("Decision Tree Model Trained Successfully!")
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 119

Markdown Content:

This code trains a Decision Tree Regressor model using the training dataset. The model learns patterns from the input features (X_train) to predict the target variable (y_train). The random state ensures reproducibility, and the message confirms that the model has been successfully trained for regression tasks.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 120

Code:

```
dt_train_pred = decision_tree.predict(X_train)

dt_test_pred = decision_tree.predict(X_test)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 121

Code:

```
dt_mae = mean_absolute_error(y_test, dt_test_pred)

dt_mse = mean_squared_error(y_test, dt_test_pred)

dt_rmse = np.sqrt(dt_mse)

dt_r2 = r2_score(y_test, dt_test_pred)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 122

Markdown Content:

This code generates predictions using the trained Decision Tree Regressor model. It predicts values for both the training dataset (X_train) and the testing dataset (X_test). These predictions are later used to evaluate the model's performance and check how well it generalizes to unseen data.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 123

Code:

```
dt_train_score = decision_tree.score(X_train, y_train)

dt_test_score = decision_tree.score(X_test, y_test)

print("Training Score :", dt_train_score)

print("Testing Score  :", dt_test_score)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 124

Markdown Content:

This code evaluates the performance of the Decision Tree Regressor by calculating R^2 scores on both training and testing data. The training score shows how well the model fits the training data, while the testing score indicates how well it generalizes to unseen data. Comparing both scores helps identify whether the model is overfitting or performing well on new data.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 125

Code:

```
from sklearn.model_selection import GridSearchCV

parameters = {

    "max_depth": [5,10,15,20,None],

    "min_samples_split": [2,5,10],

    "min_samples_leaf": [1,2,4]

}

dt_grid = GridSearchCV(

    estimator=DecisionTreeRegressor(random_state=42),

    param_grid=parameters,

    cv=5,

    scoring="r2",

    n_jobs=-1

)

dt_grid.fit(X_train, y_train)

print("Best Parameters")

print(dt_grid.best_params_)

print("\nBest CV Score")

print(dt_grid.best_score_)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 126**Markdown Content:**

This code performs hyperparameter tuning of a Decision Tree Regressor using GridSearchCV. It tests different combinations of parameters like max_depth, min_samples_split, and min_samples_leaf using 5-fold cross-validation. The model selects the best parameter combination based on the highest R² score. Finally, it outputs the best parameters and the best cross-validation score, helping to improve model performance and reduce overfitting.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 127

Code:

```
best_dt = dt_grid.best_estimator_  
  
best_dt_pred = best_dt.predict(X_test)  
  
best_dt_mae = mean_absolute_error(y_test,best_dt_pred)  
  
best_dt_mse = mean_squared_error(y_test,best_dt_pred)  
  
best_dt_rmse = np.sqrt(best_dt_mse)  
  
best_dt_r2 = r2_score(y_test,best_dt_pred)  
  
print("Best Decision Tree Results")  
  
print("MAE :",best_dt_mae)  
  
print("MSE :",best_dt_mse)  
  
print("RMSE :",best_dt_rmse)  
  
print("R² :",best_dt_r2)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 128

Markdown Content:

This code evaluates the best Decision Tree model (after GridSearchCV tuning) on the test dataset. It makes predictions and calculates key performance metrics including MAE, MSE, RMSE, and R² score. These metrics help measure prediction error and overall model accuracy, where lower error values and a higher R² score indicate better model performance and generalization.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 129

Code:

```
from sklearn.ensemble import RandomForestRegressor
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 130

Code:

```
random_forest = RandomForestRegressor(  
  
    n_estimators=100,  
  
    random_state=42,  
  
    n_jobs=-1  
  
)  
  
random_forest.fit(X_train,y_train)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 131

Markdown Content:

This code trains a Random Forest Regressor using 100 decision trees to improve prediction accuracy. The model is fitted on the training data, where multiple trees work together to reduce overfitting and improve generalization. The `random_state` ensures reproducibility, and `n_jobs=-1` uses all CPU cores for faster training.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 132

Code:

```
rf_train_pred = random_forest.predict(X_train)  
  
rf_test_pred = random_forest.predict(X_test)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 133

Markdown Content:

This code generates predictions using the trained Random Forest Regressor model. It predicts output values for both the training data and the testing data. These predictions are later used to evaluate model performance and check how well the model generalizes to unseen data.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 134

Code:

```
rf_mae = mean_absolute_error(y_test, rf_test_pred)
```

```
rf_mse = mean_squared_error(y_test, rf_test_pred)
```

```
rf_rmse = np.sqrt(rf_mse)
```

```
rf_r2 = r2_score(y_test, rf_test_pred)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 135

Markdown Content:

This code evaluates the Random Forest Regressor model using important regression metrics such as MAE, MSE, RMSE, and R² score. These metrics measure the prediction error and overall accuracy of the model on the test dataset. Lower error values and a higher R² score indicate better model performance and stronger generalization ability.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 136

Code:

```
rf_train_score = random_forest.score(X_train,y_train)

rf_test_score = random_forest.score(X_test,y_test)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 137

Markdown Content:

This code calculates the R^2 scores of the Random Forest model on both training and testing data. The training score shows how well the model fits the training dataset, while the testing score indicates how well it generalizes to unseen data. Comparing both scores helps identify whether the model is overfitting, underfitting, or performing well overall.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 138

Code:

```
importance = pd.DataFrame({

    "Feature":X_train.columns,

    "Importance":random_forest.feature_importances_

})

importance = importance.sort_values(

    by="Importance",

    ascending=False

)

importance
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 139

Markdown Content:

This code extracts and displays the feature importance from the Random Forest model. It creates a DataFrame showing each feature and its contribution to the model's predictions, then sorts them in descending order of importance. This helps identify which features have the most impact on the target variable and improves model interpretability and feature selection.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 140

Code:

```
plt.figure(figsize=(10,6))

sns.barplot(
    data=importance.sort_values("Importance"),
    x="Importance",
    y="Feature",
    palette="viridis"    # nice gradient color
)

plt.title("Random Forest Feature Importance")
plt.grid(axis='x', linestyle='--', alpha=0.4)

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 141

Markdown Content:

This code visualizes the feature importance of the Random Forest model using a bar plot. It displays how much each feature contributes to the model's predictions in descending order. This helps in understanding which variables are most influential and improves model interpretability and feature selection.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 142

Code:

```
tree_models = pd.DataFrame({
    "Model":[
        "Decision Tree",
        "Random Forest"
    ],
    "MAE":[
        best_dt_mae,
        rf_mae
    ],
    "MSE":[
        best_dt_mse,
        rf_mse
    ],
    "RMSE":[
        best_dt_rmse,
        rf_rmse
    ],
    "R2 Score":[
        best_dt_r2,
        rf_r2
    ]
})
```

tree_models

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 143

Markdown Content:

This code used to model mae mse and rsme and r2 score.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 144

Code:

```
plt.figure(figsize=(8,5))

plt.bar(
    tree_models["Model"],
    tree_models["R2 Score"],
    color=["tomato", "seagreen"],
    edgecolor="black",
    width=0.5
)

plt.title("Decision Tree vs Random Forest (R2 Score)")
plt.ylabel("R2 Score")

plt.ylim(0, 1)

plt.grid(axis='y', linestyle='--', alpha=0.4)

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 145

Markdown Content:

This code used to compare Decision Tree and Random forset using barplot.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 146

Code:

```
from sklearn.svm import SVR

print("SVR Imported Successfully!")
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 147

Markdown Content:

Import Svr using the library to used in machine learning .

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 148

Code:

```
sample_size = 20000

X_train_svr = X_train_scaled[:sample_size]

y_train_svr = y_train.iloc[:sample_size]

print("Training Samples:", X_train_svr.shape)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 149

Markdown Content:

this code used to test taining sample data (3360,11)

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 150

Code:

```
linear_svr = SVR(  
    kernel="linear",  
    C=1.0,  
    epsilon=0.1  
)  
  
linear_svr.fit(X_train_svr, y_train_svr)  
  
print("Linear SVR Trained Successfully!")
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 151

Markdown Content:

this code used to linear svr trained successfully

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 152

Code:

```
linear_pred = linear_svr.predict(X_test_scaled)  
  
linear_mae = mean_absolute_error(y_test, linear_pred)  
  
linear_mse = mean_squared_error(y_test, linear_pred)  
  
linear_rmse = np.sqrt(linear_mse)  
  
linear_r2 = r2_score(y_test, linear_pred)  
  
print(f"MAE      : {linear_mae:.2f}")  
print(f"MSE      : {linear_mse:.2f}")  
print(f"RMSE     : {linear_rmse:.2f}")  
print(f"R² Score : {linear_r2:.4f}")
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 153

Markdown Content:

This code evaluates the Linear Support Vector Regressor (SVR) model on the test dataset by calculating key performance metrics: MAE, MSE, RMSE, and R^2 score. These metrics help measure prediction error and model accuracy. Lower error values and a higher R^2 score indicate better model performance and generalization on unseen data.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 154

Code:

```
rbf_svr = SVR(  
    kernel="rbf",  
    C=10,  
    gamma="scale",  
    epsilon=0.1  
)  
  
rbf_svr.fit(X_train_svr, y_train_svr)  
  
print("RBF SVR Trained Successfully!")
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 155

Markdown Content:

This code trains a Support Vector Regression (SVR) model with RBF kernel, which helps capture non-linear relationships in the data. The model is fitted on the training dataset using tuned hyperparameters like C, gamma, and epsilon. The message confirms that the RBF SVR model has been successfully trained.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 156

Code:

```
rbf_pred = rbf_svr.predict(X_test_scaled)

rbf_mae = mean_absolute_error(y_test, rbf_pred)

rbf_mse = mean_squared_error(y_test, rbf_pred)

rbf_rmse = np.sqrt(rbf_mse)

rbf_r2 = r2_score(y_test, rbf_pred)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 157

Markdown Content:

This code evaluates the RBF Kernel SVR model on the test dataset by making predictions and calculating key performance metrics such as MAE, MSE, RMSE, and R^2 score. These metrics help measure prediction error and accuracy. Lower error values and a higher R^2 score indicate better model performance and strong generalization on unseen data.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 158

Code:

```
svr_parameters = {

    "C": [0.1, 1, 10],

    "gamma": ["scale", "auto"],

    "epsilon": [0.1, 0.5, 1]

}

svr_grid = GridSearchCV(

    estimator=SVR(kernel="rbf"),
```

```

    param_grid=svr_parameters,

    cv=3,

    scoring="r2",

    n_jobs=-1

)

svr_grid.fit(X_train_svr, y_train_svr)

print("Best Parameters")

print(svr_grid.best_params_)

print("\nBest CV Score")

print(svr_grid.best_score_)

```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 159

Markdown Content:

This code performs hyperparameter tuning for an RBF Support Vector Regression (SVR) model using GridSearchCV. It tests different combinations of parameters like C, gamma, and epsilon using 3-fold cross-validation. The best parameter set is selected based on the highest R^2 score, and the best cross-validation score is displayed. This helps improve model performance and ensure better generalization on unseen data.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 160

Code:

```

best_svr = svr_grid.best_estimator_

best_pred = best_svr.predict(X_test_scaled)

best_mae = mean_absolute_error(y_test,best_pred)

best_mse = mean_squared_error(y_test,best_pred)

```

```
best_rmse = np.sqrt(best_mse)

best_r2 = r2_score(y_test,best_pred)

print("===== Best SVR =====")

print("MAE :",best_mae)

print("MSE :",best_mse)

print("RMSE :",best_rmse)

print("R² :",best_r2)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 161

Markdown Content:

This code evaluates the best-tuned SVR model (after GridSearchCV) on the test dataset. It makes predictions and calculates performance metrics such as MAE, MSE, RMSE, and R² score. These metrics help measure prediction error and accuracy, where lower error values and a higher R² score indicate better model performance and strong generalization ability.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 162

Code:

```
plt.scatter(y_test, best_pred, alpha=0.5, c=y_test, cmap="viridis")
plt.colorbar(label="Actual Price")
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 163

Markdown Content:

This code visualizes the performance of the best SVR model by plotting a scatter graph between actual and predicted values. It helps assess how closely the model's predictions match the real values. If the points are closer to a straight diagonal line, it indicates better model accuracy and strong predictive performance.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 164

Code:

```
residuals = y_test - best_pred

plt.figure(figsize=(8,6))

sns.scatterplot(
    x=best_pred,
    y=residuals,
    color="green"    # change color here
)

plt.axhline(0, color="red", linestyle="--")

plt.title("Residual Plot (Best SVR)")
plt.xlabel("Predicted Price")
plt.ylabel("Residual")

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 165

Markdown Content:

This code evaluates the RBF SVR model predictions using standard regression metrics such as MAE, MSE, RMSE, and R^2 score. These metrics measure the prediction error and accuracy of the model on the test data. Lower error values and a higher R^2 score indicate better model performance and stronger generalization on unseen data.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 166

Code:

```
svr_results = pd.DataFrame({  
    "Model":[  
        "Linear SVR",  
        "RBF SVR",  
        "Best Tuned SVR"  
    ],  
    "MAE":[  
        linear_mae,  
        rbf_mae,  
        best_mae  
    ],  
    "RMSE":[  
        linear_rmse,  
        rbf_rmse,  
        best_rmse  
    ],  
    "R² Score":[  
        linear_r2,  
        rbf_r2,  
        best_r2  
    ]  
})
```

svr_results

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 167

Markdown Content:

This code creates a comparison table for different SVR models including Linear SVR, RBF SVR, and the best tuned SVR. It organizes key evaluation metrics such as MAE, RMSE, and R^2 score into a single DataFrame. This helps easily compare model performance and identify the best-performing SVR model based on accuracy and error metrics.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 168

Code:

```
plt.figure(figsize=(8,5))

plt.bar(
    svr_results["Model"],
    svr_results["R2 Score"],
    color=["red", "green", "blue"]
)

plt.title("SVR Performance Comparison")
plt.ylabel("R2 Score")
plt.xticks(rotation=10)

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 169

Markdown Content:

```
## SVR Performance Comparison
```

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 170

Code:

```
model_results = pd.DataFrame({

    "Model":[
        "Ridge Regression",
        "Lasso Regression",
        "Decision Tree",
        "Random Forest",
        "Linear SVR",
        "Best Tuned SVR"
    ],

    "MAE":[
        ridge_mae_best,
        lasso_mae_best,
        best_dt_mae,
        rf_mae,
        linear_mae,
        best_mae
    ],

    "MSE":[
        ridge_mse_best,
        lasso_mse_best,
        best_dt_mse,
        rf_mse,
        linear_mse,
        best_mse
    ],

    "RMSE":[
        ridge_rmse_best,
        lasso_rmse_best,
        best_dt_rmse,
        rf_rmse,
        linear_rmse,
        best_rmse
    ],

    "R2 Score":[
        ridge_r2_best,
        lasso_r2_best,
        best_dt_r2,
        rf_r2,
        linear_r2,
        best_r2
    ]

})

model_results
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 171

Markdown Content:

This code creates a comparison table for different SVR models including Linear SVR, RBF SVR, and the best tuned SVR. It organizes key evaluation metrics such as MAE, RMSE, and R^2 score into a single DataFrame. This helps easily compare model performance and identify the best-performing SVR model based on accuracy and error metrics.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 172

Code:

```
model_results = model_results.sort_values(  
    by="R2 Score",  
    ascending=False  
)  
  
model_results
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 173

Markdown Content:

This code sorts the model performance results based on the R^2 score in descending order. It helps identify the best-performing model at the top of the table, making it easier to compare different models and select the one with the highest accuracy and best generalization on the dataset.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 174

Code:

```
best_model = model_results.iloc[0]

print("Best Performing Model")

print(best_model)
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 175

Markdown Content:

This code selects the best-performing model from the sorted results table by taking the first row (highest R^2 score). It then prints all details of that model, including its evaluation metrics. This helps in final model selection based on overall performance and accuracy.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 176

Code:

```
plt.figure(figsize=(10,6))

sns.barplot(
    data=model_results,
    x="Model",
    y="R2 Score",
    palette=["red", "green", "blue", "orange"]
)

plt.xticks(rotation=20)
plt.title("Model Comparison using  $R^2$  Score")

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 177

Markdown Content:

Model Comparison using R^2 Score

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 178

Markdown Content:

This code compare all models together using barplot.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 179

Code:

```
plt.figure(figsize=(10,6))

sns.barplot(
    data=model_results,
    x="Model",
    y="RMSE",
    palette="magma"
)

plt.xticks(rotation=20)
plt.title("Model Comparison using RMSE")

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 180

Markdown Content:

Model Comparison using RMSE

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 181

Markdown Content:

This code compare all models together using barplot.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 182

Code:

```
plt.figure(figsize=(10,6))

sns.barplot(
    data=model_results,
    x="Model",
    y="MAE",
    palette="crest"
)

plt.xticks(rotation=20)
plt.title("Model Comparison using MAE")

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 183

Markdown Content:

Model Comparison using MAE

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 184

Markdown Content:

This code compare all models together using barplot.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 185

Code:

```
plt.figure(figsize=(10,6))

sns.barplot(
    data=model_results,
    x="Model",
    y="MSE",
    palette=["red", "green", "blue", "orange"]
)

plt.xticks(rotation=20)
plt.title("Model Comparison using MSE")

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 186

Markdown Content:

```
## Model Comparison using MSE
```

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 187

Markdown Content:

This code compare all models together using barplot.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 188

Code:

```
train_test = pd.DataFrame({
    "Model": [
        "Ridge",
        "Lasso",
        "Decision Tree",
```

```

        "Random Forest"
    ],
    "Training Score":[
        best_ridge.score(X_train_scaled,y_train),
        best_lasso.score(X_train_scaled,y_train),
        best_dt.score(X_train,y_train),
        random_forest.score(X_train,y_train)
    ],
    "Testing Score":[
        ridge_r2_best,
        lasso_r2_best,
        best_dt_r2,
        rf_r2
    ]
})

train_test

```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 189

Markdown Content:

This code used to compare 0 Ridge

- 1 Lasso
- 2 Decision Tree
- 3 Random Forest

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 190

Code:

```

train_test.set_index("Model").plot(
    kind="bar",
    figsize=(10,6)
)

plt.title("Training Score vs Testing Score")

```

```
plt.ylabel("R2 Score")

plt.xticks(rotation=0)

plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 191

Markdown Content:

This code visualizes and compares the training and testing R^2 scores of different models using a bar plot. It helps quickly identify how each model performs on both seen and unseen data, making it easier to detect overfitting, underfitting, or good generalization.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 192

Code:

```
for i in range(len(train_test)):

    train = train_test.iloc[i]["Training Score"]
    test = train_test.iloc[i]["Testing Score"]

    gap = train - test

    print("="*50)
    print("Model :", train_test.iloc[i]["Model"])

    print("Training :", round(train,4))
    print("Testing  :", round(test,4))

    if gap > 0.10:
        print("Status : Overfitting")

    elif train < 0.70 and test < 0.70:
        print("Status : Underfitting")

    else:
        print("Status : Good Generalization")
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 193

Markdown Content:

This code evaluates multiple machine learning models by comparing their training and testing scores. It calculates the performance gap to identify whether each model is overfitting, underfitting, or generalizing well. If the training score is much higher than the testing score, the model is considered overfitting. If both scores are low, it indicates underfitting. Otherwise, the model is considered to have good generalization performance.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 194

Code:

```
plt.figure(figsize=(10,6))

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

sns.barplot(
    data=importance,
    x="Importance",
    y="Feature",
    palette="mako"
)

plt.title("Feature Importance - Random Forest")
plt.tight_layout()
plt.show()
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 195

Markdown Content:

This code visualizes the feature importance of a Random Forest model using a bar plot. It first sorts the features based on their importance scores and then displays them in descending order. The plot helps identify which features have the most impact on the model's predictions, making the model more interpretable and useful for feature selection.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 196

Code:

```
model_results.to_csv(
    "Model_Comparison.csv",
    index=False
)

print("Results Saved Successfully!")
```

Purpose:

This code cell performs the operation defined in the notebook workflow. It contributes to data loading, preprocessing, visualization, model building, evaluation, or comparison depending on its position in the pipeline.

Working:

The code is executed sequentially as part of the machine learning workflow. Variable names, functions, and transformations are preserved exactly as implemented in the notebook.

STEP 197

Markdown Content:

Conclusion:

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

STEP 198

Markdown Content:

This project successfully implemented and compared multiple machine learning regression models for house price prediction. Through data preprocessing, exploratory data analysis, feature engineering, regularization, hyperparameter tuning, and cross-validation, reliable predictive models were developed and evaluated. Based on the evaluation metrics, Random Forest Regression (or your actual best model) achieved the highest accuracy and best generalization performance. The project demonstrates the effectiveness of machine learning in solving real-world real estate prediction problems and provides a strong foundation for future deployment using real-world datasets and web-based applications.

Interpretation:

This markdown cell documents the reasoning, explanation, or observations associated with the surrounding code cells.

Project Summary

The notebook implements an end-to-end house price prediction workflow: dataset loading, exploratory data analysis, missing-value and duplicate checks, correlation and distribution analysis, feature/target separation, train-test splitting, and feature scaling. It then trains and evaluates multiple regression approaches — Ridge Regression, Lasso Regression (both with GridSearchCV hyperparameter tuning), cross-validation (K-Fold, Stratified K-Fold, Time Series Split), Decision Tree Regression (tuned), Random Forest Regression, and Support Vector Regression (Linear and RBF kernels, tuned via GridSearchCV). Model performance is assessed using MAE, MSE, RMSE, and R^2 Score, with training-vs-testing comparisons used to check for overfitting or underfitting. Feature importance from the Random Forest model is visualized, all models are compared side by side, and the best-performing model is identified and the comparison results are exported to CSV.